

Fast Parallel Hypertree Decompositions in Logarithmic Recursion Depth

SIGMOD-SIGACT-SIGAI Symposium
on Principles of Database Systems 2022
June 15, Philadelphia

Georg Gottlob^{1,2} Matthias Lanzinger¹ **Cem Okulmus**² Reinhard Pichler²

¹University of Oxford ²TU Wien

- A quick overview over the setting and motivation

THREE PROBLEMS

- Conjunctive Query Evaluation
- Constraint Satisfaction Problem
- Homomorphism Problem

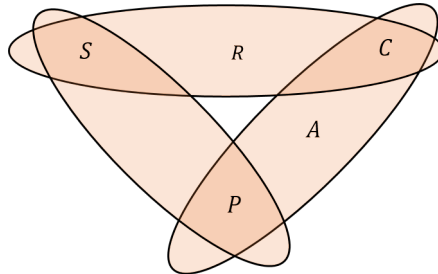
All these problems are essentially the same

All these problems are based on hypergraphs.

Query

Is there any teacher having a child enrolled in her course?

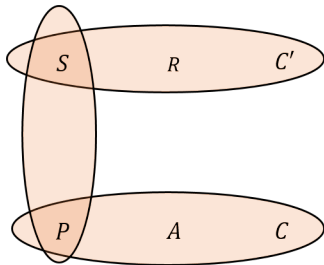
$$ans \leftarrow Enrolled(S, C, R) \wedge Teaches(P, C, A) \wedge Parent(P, S)$$



Query

Is there any teacher having a child enrolled in **some** course ?

$$ans \leftarrow Enrolled(S, C', R) \wedge Teaches(P, C, A) \wedge Parent(P, S)$$



ACYCLIC CQs (CONT.)

Query

Is there any teacher having a child enrolled in **some** course ?

$$ans \leftarrow Enrolled(S, C', R) \wedge Teaches(P, C, A) \wedge Parent(P, S)$$

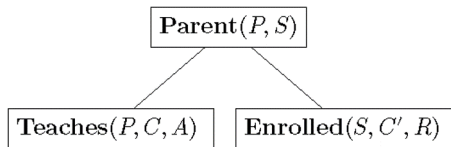
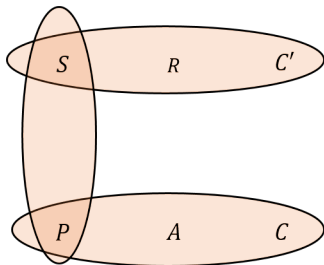


Figure 1: Join Tree

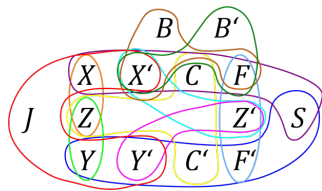
- **NP-complete in the general case**
(Chandra and Merlin '77)
- **Tractable in case of acyclic CQs**
(Yannakakis '81)

Interest in larger tractable classes in CQs

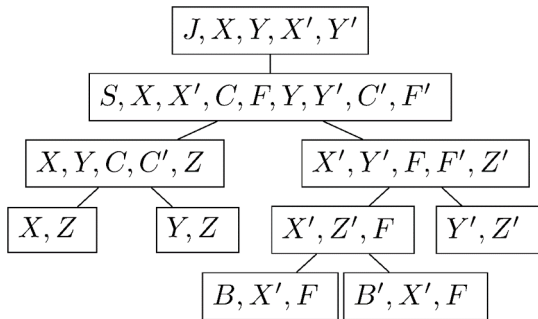
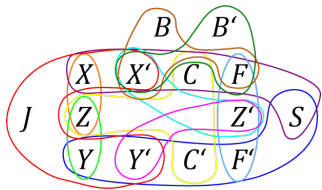
$$\begin{aligned} ans \leftarrow & a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge \\ & g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F) \end{aligned}$$

TREE DECOMPOSITION (CONT.)

$$\begin{aligned} \text{ans} \leftarrow & a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge \\ & g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F) \end{aligned}$$

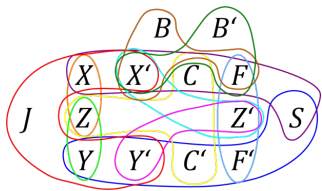


TREE DECOMPOSITION (CONT.)

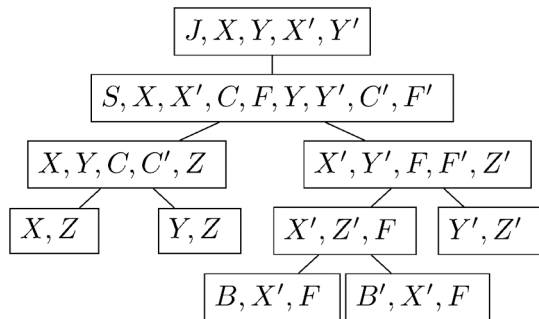
$$\begin{aligned} \text{ans} \leftarrow & a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge \\ & g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F) \end{aligned}$$


TREE DECOMPOSITION (CONT.)

$$\text{ans} \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$$

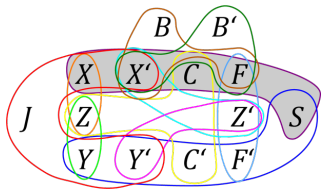


- Variables of each atom covered by some node

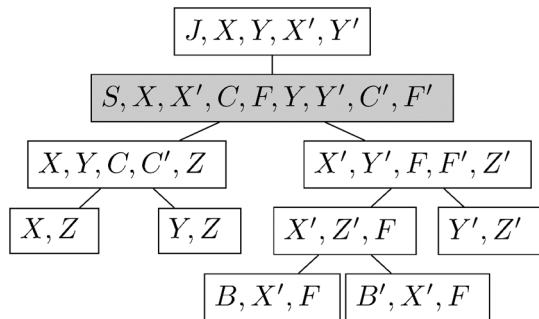


TREE DECOMPOSITION (CONT.)

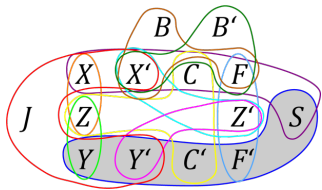
$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$



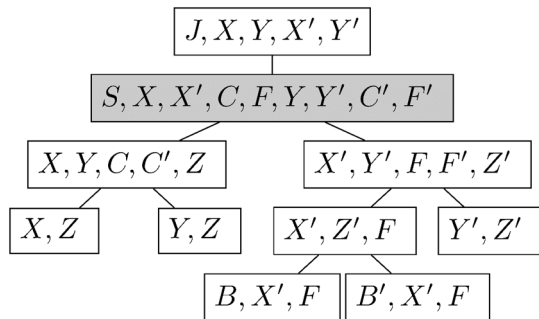
- Variables of each atom covered by some node



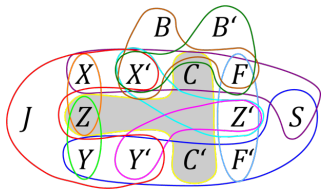
TREE DECOMPOSITION (CONT.)

$$\text{ans} \leftarrow a(S, X, X', C, F) \wedge \textcolor{red}{b(\textcolor{red}{S}, \textcolor{red}{Y}, \textcolor{red}{Y'}, \textcolor{red}{C'}, \textcolor{red}{F'})} \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$$


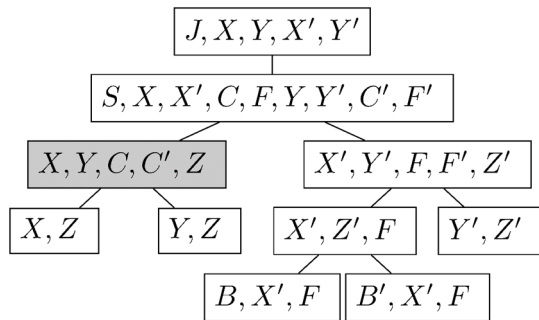
- Variables of each atom covered by some node



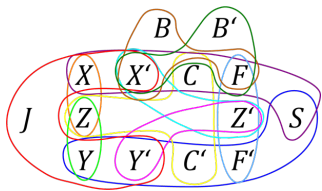
TREE DECOMPOSITION (CONT.)

$$\text{ans} \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$$


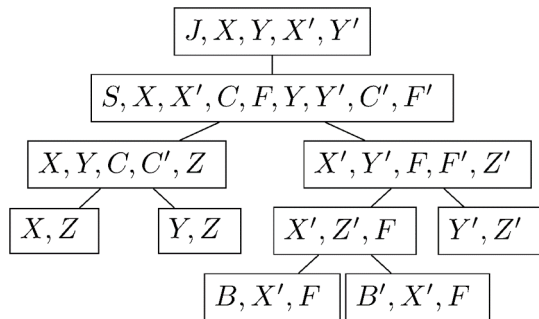
- Variables of each atom covered by some node



TREE DECOMPOSITION (CONT.)

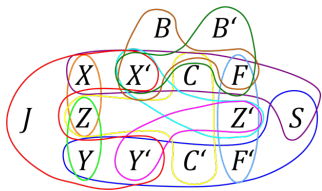
$$\text{ans} \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$$


- Variables of each atom covered by some node
- Connectedness Condition

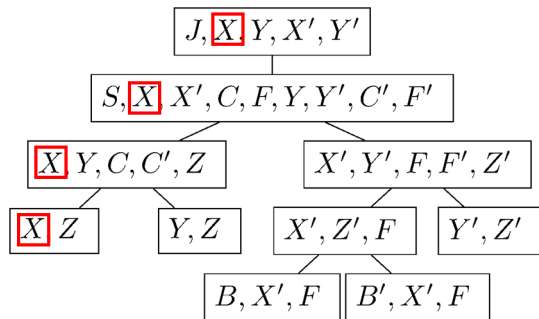


TREE DECOMPOSITION (CONT.)

$$\text{ans} \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$$

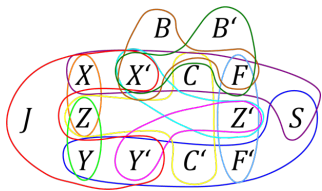


- Variables of each atom covered by some node
- Connectedness Condition

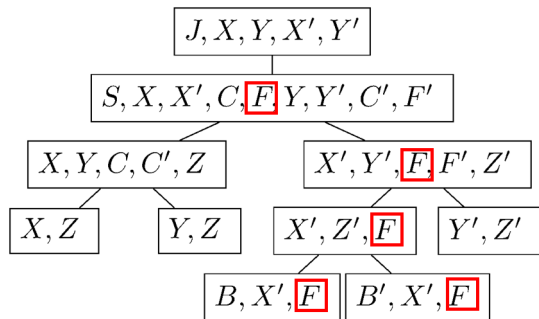


TREE DECOMPOSITION (CONT.)

$$\text{ans} \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$$

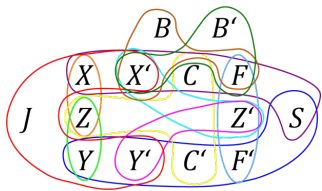


- Variables of each atom covered by some node
- Connectedness Condition

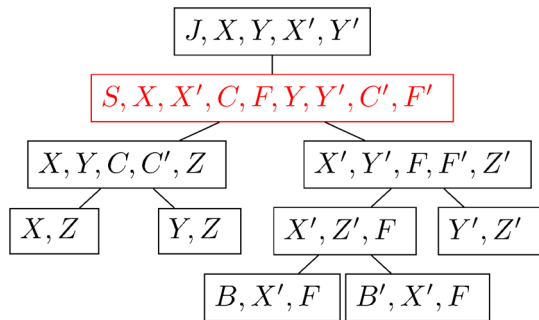


TREE DECOMPOSITION (CONT.)

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$



- Variables of each atom covered by some node
- Connectedness Condition



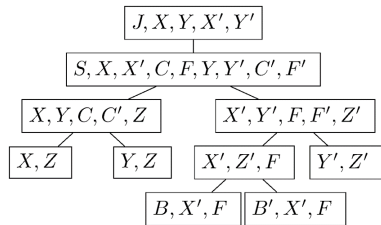
Width: 8

- Consider a set of vertices $V' \subseteq V(H)$
- An edge cover is a set of edges $E' \subseteq E(H)$, s.t.
all vertices in V' are “covered” by E' , i.e. $V' \subseteq \bigcup_{e \in E'} e$
- Add edge covers to the tree decomposition

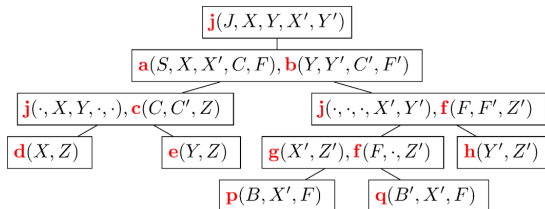
GENERALIZED HYPERTREE DECOMPOSITION (GHD)

TREE DECOMPOSITION + EDGE COVERS

$ans \leftarrow a(S, X, X', C, F) \wedge b(S, Y, Y', C', F') \wedge c(C, C', Z) \wedge d(X, Z) \wedge e(Y, Z) \wedge f(F, F', Z') \wedge g(X', Z') \wedge h(Y', Z') \wedge j(J, X, Y, X', Y') \wedge p(B, X', F) \wedge q(B', X', F)$



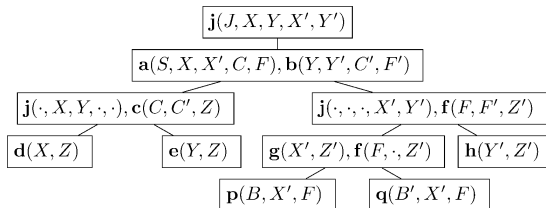
Width: 8



Generalized Hypertree Width (ghw): 2

HYPERTREE DECOMPOSITIONS

GHD + SPECIAL CONDITION

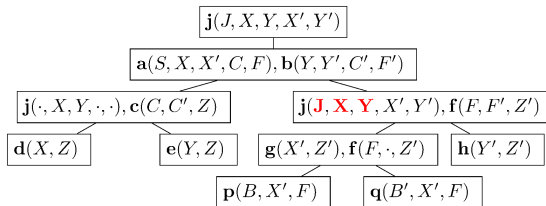


Generalized Hypertree Width (hw): 2

HYPERTREE DECOMPOSITIONS

GHD + SPECIAL CONDITION

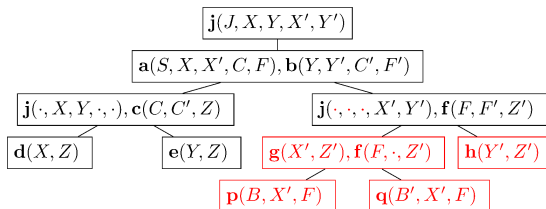
- Each variable that **disappeared** at some node **n**



HYPERTREE DECOMPOSITIONS

GHD + SPECIAL CONDITION

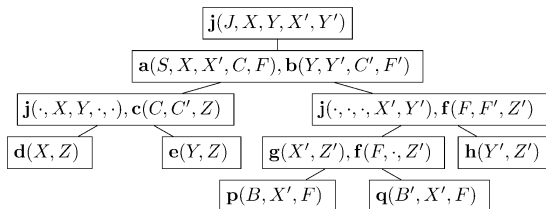
- Each variable that **disappeared** at some node **n**
- does **not reappear** in the subtree rooted at **n**



HYPERTREE DECOMPOSITIONS

GHD + SPECIAL CONDITION

- Each variable that **disappeared** at some node **n**
- does **not reappear** in the subtree rooted at **n**



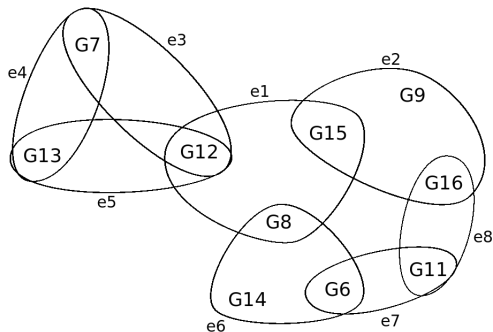
Hypertree Width (hw): 2

- We can use the HD to solve CQs in polynomial time
 - More precisely, in time $O(n^k)$, where n is size of the database and k the width of HD.

Yannakakis' algorithm with HDs

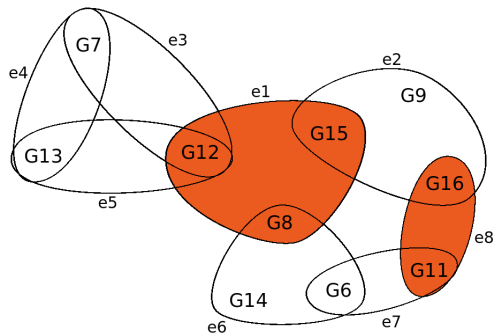
1. Join all tables that occur in edge covers, creating a new equivalent but acyclic query
2. Now we treat the HD as a join tree and perform Yannakakis' algorithm

TOP-DOWN HYPERGRAPH DECOMPOSITION



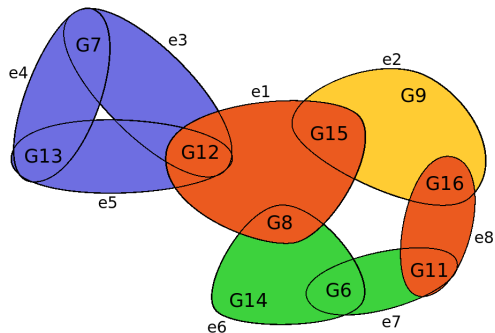
- Finding a cut (separation)

e1, e8



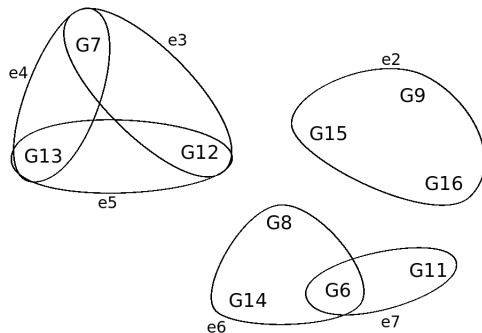
- Finding a cut (separation)

e1, e8



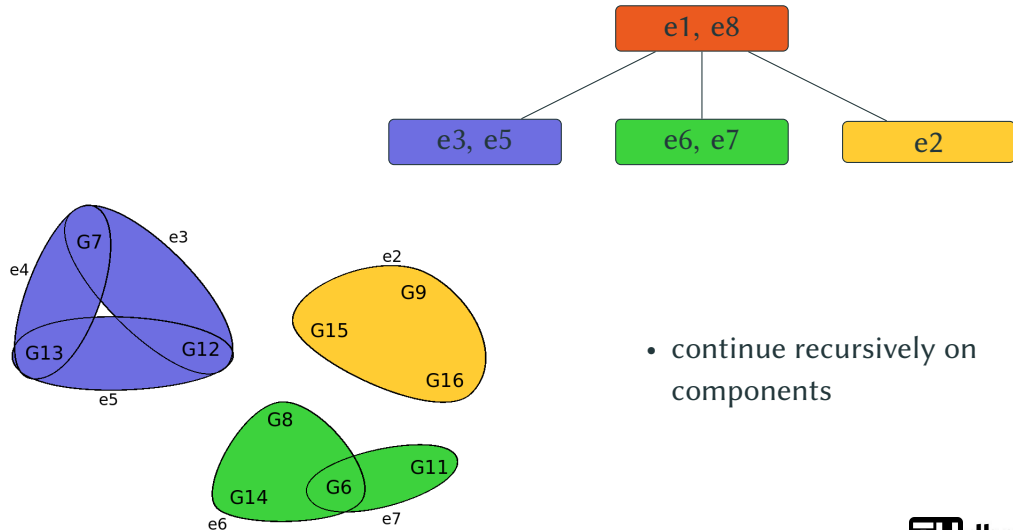
- new connected components

e1, e8



- continue recursively on components

TOP-DOWN HYPERGRAPH DECOMPOSITION



Balanced Separator

For a hypergraph H and a set S of edges, we call S a **balanced separator** if all components of H when separated by S have size $\leq E(H)/2$.

Observation [Adler, Gottlob, Grohe 2007]

Given an HD (likewise a GHD) T of width k for a hypergraph H , then T contains a node p such that $\lambda(p)$ is a **balanced separator**.

OUR GOAL & OUR CHALLENGES

Goal

- Use balanced separators to design a parallel algorithm for computing HDs

Challenges

- Getting the bags from the edge covers alone
- HDs are rooted: how to connect subproblems up and down the tree?

- Challenge: We cannot afford to guess all possible bags (exponentially many)

- Challenge: We cannot afford to guess all possible bags (exponentially many)
- Solution: Using some technical lemmas from [Gottlob, Leone, Scarcello; 2002], we show the following:

$$\chi_c = \bigcup \lambda_c \cap (C \cup \bigcup \lambda_p)$$

where c is the node we want to determine, p is its parent node, and C is the component of p that c covers.

- Challenge: We cannot afford to guess all possible bags (exponentially many)
- Solution: Using some technical lemmas from [Gottlob, Leone, Scarcello; 2002], we show the following:

$$\chi_c = \bigcup \lambda_c \cap (C \cup \bigcup \lambda_p)$$

where c is the node we want to determine, p is its parent node, and C is the component of p that c covers.

⇒ If we guess the edge cover of the parent p as well, we can determine the exact bag of the child c .

- Challenge: We cannot afford to guess all possible bags (exponentially many)
- Solution: Using some technical lemmas from [Gottlob, Leone, Scarcello; 2002], we show the following:

$$\chi_c = \bigcup \lambda_c \cap (C \cup \bigcup \lambda_p)$$

where c is the node we want to determine, p is its parent node, and C is the component of p that c covers.

⇒ If we guess the edge cover of the parent p as well, we can determine the exact bag of the child c .

- Caveat: This actually requires us to define a new normal form for HDs, and a bit of technical groundwork to prove correctness

- Challenge: We want to be able to jump into the “middle” of an HD

- Challenge: We want to be able to jump into the “middle” of an HD
- Solution: We introduce an *extended subhypergraph* definition
- We define notions of component, balanced separator on this new type of hypergraph
- We define that HDs of this type of hypergraph need to satisfy certain conditions:
 - The root needs to cover a set of vertices **Conn** (connection above)
 - There need to nodes that cover some **special edges** (connection(s) below)

- We presented the algorithm, log- k -decomp
 - parallel algorithm for solving HDs, with a log. bound on the depth of recursive calls
- Implementation available under github.com/cem-okulmus/log-k-decomp
- We now managed to solve 85 % of HyberBench optimally for HDs.
 - Of all known instances with width at most 6, log- k -decomp solves 92 % optimally.
 - The average running time across all tests was 30.5 sec, compared to 280.2 from the next best solver compared.
 - For many instances, we can find optimal decompositions often in just a few milliseconds
- We have shown that it is feasible to find decompositions of small width for almost all CSPs and CQs in feasible time

- The next goal lies in extending existing CSP solvers or query engines to utilise structural properties, using hypergraph decompositions in some form
 - The goal is to somehow combine structural methods with existing quantitative ones
 - We are currently working on this
- Call To The Community: We are looking for **join queries** which are challenging for existing RDBMSs
 - We are looking for real-world queries (and datasets)
 - The difficulty should be on the processing of joins, not in aggregates or subexpressions.
 - Our goal: creating a benchmark of real-world queries that are really hard to solve